

Loop Subdivision Surfaces

Algorithm Description

The Loop subdivision algorithm operates by splitting each triangle into four smaller triangles and repositioning vertices according to specific masks. The algorithm distinguishes between two types of vertices:

1 Even Vertices (Original Vertices)

Even vertices are the original vertices from the input mesh. Their new positions are computed as a weighted combination of their current position and the positions of their neighbors.

For interior vertices with valence n (number of neighboring vertices):

$$v' = (1 - n \cdot \alpha) \cdot v + \alpha \cdot \sum(\text{neighbors})$$

Where the weight α is computed using Loop's formula:

$$\alpha = (1/n) \times (5/8 - (3/8 + \cos(2\pi/n)/4)^2)$$

For boundary vertices (vertices on mesh edges with only one adjacent triangle):

$$v' = (3/4) \cdot v + (1/8) \cdot (\text{neighbor}_1 + \text{neighbor}_2)$$

2 Odd Vertices (New Edge Midpoints)

Odd vertices are new vertices created at the midpoint of each edge in the original mesh.

For interior edges (edges shared by two triangles):

$$v_{\text{odd}} = (3/8) \cdot (v_1 + v_2) + (1/8) \cdot (v_3 + v_4)$$

Where v_1 and v_2 are the edge endpoints, and v_3 and v_4 are the opposite vertices in the two adjacent triangles.

For boundary edges (edges belonging to only one triangle):

$$v_{\text{odd}} = (1/2) \cdot (v_1 + v_2)$$

Implementation Details

The main function `subdivideLoop()` in `Mesh.h` performs the subdivision in five distinct phases:

1. **Data Structure Initialization:** Create containers for new vertices, triangles, edge-to-vertex mappings, and adjacency information.
2. **Adjacency Computation:** Iterate through all triangles to build neighbor lists and identify which triangles share each edge.

3. **Boundary Detection:** Identify boundary edges (edges with only one adjacent triangle) and mark boundary vertices.
4. **Even Vertex Computation:** Compute new positions for all original vertices using the appropriate mask (interior or boundary).
5. **Odd Vertex Computation and Triangle Creation:** For each original triangle, create three odd vertices (if not already created) and generate four new triangles.

Key Data Structures

The implementation uses several important data structures:

- **Edge struct:** Represents an edge by its two vertex indices, sorted for consistent lookup.
- **trianglesOnEdge:** Maps each edge to the indices of triangles sharing that edge (used for boundary detection).
- **neighboringVertices:** Stores the set of neighboring vertices for each vertex (used for even vertex computation).
- **newVertexOnEdge:** Maps each edge to the index of its odd vertex (prevents duplicate vertex creation).

Additional Features

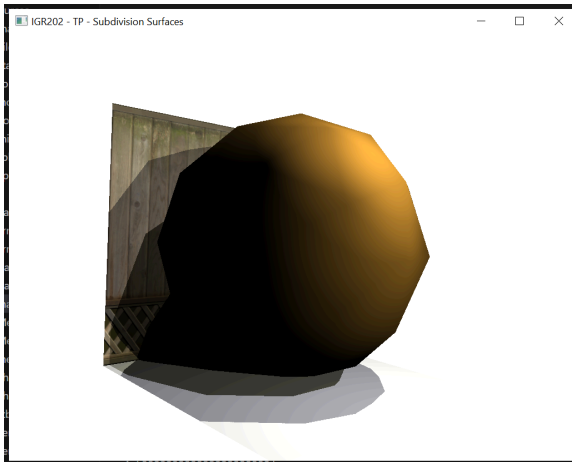
Safety Limit: Since each subdivision multiplies the triangle count by 4, the mesh grows exponentially. A safety check prevents subdivision when the mesh exceeds 200,000 triangles to avoid system freezes.

Undo Functionality: The implementation stores up to 10 previous mesh states, allowing stepping back through subdivision levels using the 'U' key.

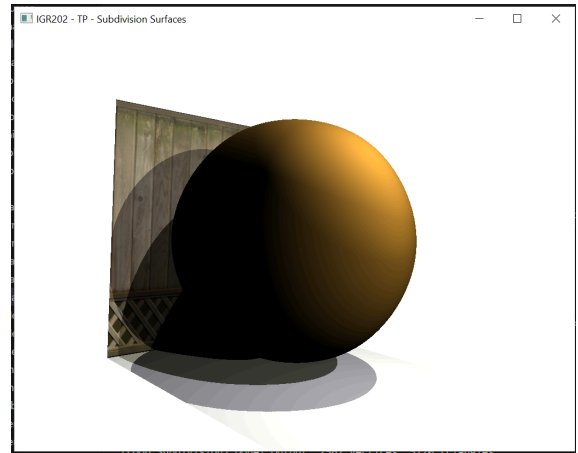
Zooming in and zooming out with scrolling is also added to observe the details and changes better

Results

The subdivision produces visually smooth results that converge toward the limit surface with each iteration.



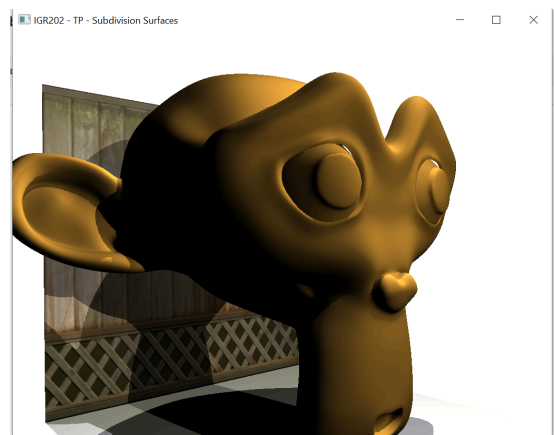
sphere before subdivision



sphere after subdivision



Monkey before subdivision



Monkey after subdivision



Comparison with Linear Subdivision

Unlike linear subdivision (which simply splits triangles at edge midpoints), Loop subdivision repositions vertices to create smooth surfaces. Linear subdivision maintains the original shape but with more triangles, while Loop subdivision approximates a smooth limit surface.

7. References

1. [Loop, C. \(1987\). Smooth Subdivision Surfaces Based on Triangles. Master's thesis, University of Utah.](#)

2. Warren, J., & Weimer, H. (2001). Subdivision Methods for Geometric Design. Morgan Kaufmann.
3. [Zorin, D., & Schröder, P. \(2000\). Subdivision for Modeling and Animation. SIGGRAPH Course Notes.](#)
4. Botsch, M., Kobbelt, L., Pauly, M., Alliez, P., & Lévy, B. (2010). Polygon Mesh Processing. A K Peters/CRC Press.
5. <http://www.holmes3d.net/graphics/subdivision/>